Reporte de Ejercicio 01

Traza de las consultas

- **AccesoNoche(Ana):** Probada (SI). Hacia adelante infiere los átomos en este orden: `Miembro(Ana)`, `Miembro(Marta)`, `Becario(Ana)`, `AccesoNoche(Ana)` y `AccesoNoche(Marta)`. Hacia atrás se comprueba usando `R_acceso` y `R_miembro_est`, basándose en sus hechos directos.
- **AccesoNoche(Luis):** No probada (NO). Solo se sabe que es estudiante, pero no tiene credencial.
- **AccesoNoche(Marta):** Probada (SI). Hacia atrás falla brevemente al evaluar la regla de estudiante, pero acierta con `R_miembro_prof`.

Respuestas a las preguntas

1. **¿Por qué `Becario(Ana)` sale en el encadenamiento hacia adelante de `AccesoNoche(Ana)` y no en el de hacia atrás?** El algoritmo hacia adelante deduce todos los hechos posibles a partir de los datos iniciales, sin importar la consulta original (es guiado por los datos). Como Ana cumple los requisitos, activa `R_beca`. En cambio, el encadenamiento hacia atrás está guiado por la meta específica (`AccesoNoche`), y como ser `Becario` no es requisito en las reglas para entrar de noche, simplemente omite evaluar esa regla.

2. **¿Por qué hacia atrás prueba `R_miembro_est` al buscar `Miembro(Marta)`, si Marta es profesora?** Porque el motor evalúa las reglas en el orden exacto en que están escritas en la base de conocimiento. `R_miembro_est` es la primera regla que concluye `Miembro(x)`. Unifica a `x` con Marta e intenta probar que es estudiante. Al fallar esa premisa específica, descarta ese camino y continúa con la siguiente regla en la lista (`R_miembro_prof`), la cual sí es exitosa.

3. **Si se añade el hecho `TieneCredencial(Luis)` y se vuelve a preguntar `AccesoNoche(Luis)`:** Aparecen los siguientes átomos nuevos durante la evaluación: `Miembro(Luis)`, `Becario(Luis)` y `AccesoNoche(Luis)`. La consulta queda **probada**.